

Relazione tecnica di laboratorio

Container escape a partire da un buffer overflow:
dalla shell nel container al controllo dell'host via socket Docker

Autore: _____ Corso di Cloud Computing / Sicurezza Data: _____

1. Scenario e obiettivo

Il laboratorio realizza, in modo riproducibile e confinato alla macchina locale, una **catena di attacco completa** che termina con un **container escape**. Il punto di partenza e' una vulnerabilita' applicativa (uno **stack buffer overflow** in un programma C) che fornisce una **shell all'interno del container**. Da quella shell, sfruttando una **misconfigurazione** (il socket del daemon Docker montato nel container), l'attaccante evade verso l'**host** e legge un file leggibile solo a livello host. L'host coinvolto e' la VM locale del Docker Engine (su Windows/macOS la VM di Docker Desktop); nessun sistema esterno e' coinvolto.

Progressione:

Applicazione vulnerabile (buffer overflow)	
-> exploit ret2win: shell nel container	[accesso al container]
-> abuso del socket Docker montato	[escape container->host]
-> lettura/scrittura come root sull'host	[condizione finale]

2. Analisi tecnica

2.1 Il meccanismo di isolamento dei container

Un container Docker e' un processo Linux dell'host isolato tramite kernel:

- **Namespaces**: vista separata di PID, filesystem/mount, rete, utenti, IPC.
- **Cgroups**: limitano e contabilizzano le risorse.
- **Capabilities**: suddividono i privilegi di root, concedendone un sottoinsieme.
- **Seccomp/AppArmor**: filtrano le system call.

Il confine di isolamento previsto e' che cio' che accade nel container non influenzi l'host. La **prima** parte della catena (buffer overflow) agisce *dentro* questo confine: fornisce esecuzione di codice nel container, ma non lo viola. Il confine viene attraversato solo nella **seconda** parte, grazie alla misconfigurazione. Elemento centrale: il daemon Docker gira come root sull'host ed espone la sua API sul socket `/var/run/docker.sock`; il socket non e' protetto da namespace e vale come controllo root sull'host.

2.2 La vulnerabilita' e la misconfigurazione

La catena combina un difetto applicativo e uno infrastrutturale.

- **Stack buffer overflow (CWE-121 / CWE-787)**. La funzione vulnerabile legge 256 byte in un buffer di 64:

```
char buffer[64];
read(0, buffer, 256); // scrittura oltre i limiti
```

Sovrascrivendo l'indirizzo di ritorno con quello della funzione `win()` (che chiama `system("/bin/sh")`), l'attaccante ottiene una shell nel container (tecnica `ret2win`). Il binario e' compilato senza stack canary e senza PIE, quindi l'indirizzo di `win()` e' fisso e l'attacco deterministico. Poiche' `system()` usa istruzioni SSE che richiedono lo stack allineato a 16 byte, il payload include un **gadget ret** per il riallineamento:

```
payload = b"A"*72 + p64(ret_gadget) + p64(win)
```

- **Montaggio del socket Docker (misconfigurazione):**

```
volumes:
- /var/run/docker.sock:/var/run/docker.sock
```

2.3 Perche' la configurazione viola il confine di isolamento

Il buffer overflow, da solo, lascia l'attaccante confinato nel container. Il montaggio del socket introduce, dentro l'ambiente isolato, un canale diretto verso il componente piu' privilegiato dell'host (il daemon root).

Dalla shell l'attaccante non rompe i namespace: usa una funzione legittima del daemon per creare un nuovo container che monta la root dell'host (-v /:/host), ottenendo lettura e scrittura su qualsiasi file dell'host. L'isolamento non viene aggirato: la configurazione lo rende irrilevante.

2.4 Assunzioni e prerequisiti

- Binario compilato senza stack canary e senza PIE (architettura x86-64; offset 72 = 64 buffer + 8 RBP salvato).
- Socket Docker montato nel container.
- Il processo nel container puo' scrivere sul socket (shell come root).
- Daemon Docker in modalita' root (non rootless); assenza di controlli sull'API.

2.5 Sequenza di sfruttamento

Baseline. Con input normale il programma stampa il saluto e 'Programma terminato normalmente.'.

Tappa 1 - buffer overflow -> shell. L'exploit ricava con nm l'indirizzo di win() e con objdump un gadget ret, poi invia 72 riempimenti + ret + win (riempito a 256 byte). Il flusso salta a win(), che avvia /bin/sh. I comandi successivi (letti dallo stdin dopo il payload) vengono eseguiti dalla shell nel container. Verifica: id restituisce uid=0(root) e viene letta la flag container-local APP-FLAG{...}.

Tappa 2a - verifica del confine. La flag di host non e' visibile dal container (non e' montata): l'isolamento e' ancora integro.

Tappa 2b - escape. Dalla shell, tramite curl sul socket, si crea un container alpine che monta la root dell'host e legge la flag di host:

```
curl -s -XPOST --unix-socket /var/run/docker.sock \
  -H "Content-Type: application/json" \
  -d '{"Image":"alpine:latest",
    "Cmd":["cat","/host/host_flag_secret.txt"],
    "Tty":true,"HostConfig":{"Binds":["/:/host"]}}' \
  http://localhost/containers/create # poi /start, /wait, /logs
```

La comparsa di HOST-FLAG{...}, assente alla tappa 2a, dimostra l'escape.

2.6 Impatto di sicurezza

- Lettura di qualsiasi file dell'host (credenziali, chiavi private).
- Scrittura sull'host (backdoor, persistenza, modifica di binari).
- Compromissione di tutti gli altri container gestiti dal daemon.
- Escalation completa: da una singola vulnerabilita' applicativa al controllo totale del nodo.

2.7 Mitigazioni e hardening

- **Non montare il socket Docker** nei container; se indispensabile, proxy API con permessi minimi e in sola lettura.
- **Docker rootless:** riduce drasticamente l'impatto di un accesso al socket.
- **Correggere il buffer overflow:** leggere entro i limiti del buffer e mantenere le protezioni del compilatore:

```
read(0, buffer, sizeof(buffer) - 1);
# compilare con stack canary (default), PIE, NX, -D_FORTIFY_SOURCE=2
```

- **Least privilege nel container:** utente non root (USER), --cap-drop=ALL, --read-only, --security-opt=no-new-privileges.
- **Difesa in profondita':** ASLR attivo, seccomp/AppArmor, user namespace remapping, scanner di misconfigurazioni (Docker Bench, Trivy).

3. Uso di vulnerabilita' note (CVE)

Escape (tappa 2). Riferimento autorevole della stessa classe: **CVE-2019-5736**, escape del runtime runc (un processo nel container sovrascrive il binario runc dell'host). Fonti in references.md (NVD, Openwall). Non e' riprodotta perche' richiede una versione vulnerabile e specifica di runc, poco riproducibile; la misconfigurazione del socket raggiunge lo stesso obiettivo (escape container->host) in modo riproducibile al 100% e realistico (comune in CI/CD e container di monitoraggio). Differenza: la CVE sfrutta un bug del runtime, il laboratorio una configurazione che annulla l'isolamento.

Vulnerabilita' applicativa (tappa 1). Il buffer overflow appartiene alle classi CWE-121/CWE-787. Istanze reali della stessa classe: **CVE-2015-7547** (stack overflow in getaddrinfo() della glibc, RCE remoto) e **CVE-2021-3156** ("Baron Samedit", overflow in sudo, escalation locale a root). Il laboratorio isola in forma minima e deterministica lo stesso meccanismo (scrittura oltre i limiti e dirottamento del flusso), senza dipendere da versioni vulnerabili di software reale.

4. Catena di attacco completa (bonus)

Tappa	Azione	Confine varcato
1. Exploit app	Buffer overflow ret2win -> system("/bin/sh")	Confine applicativo: da input non fidato a esecuzione di codice
2. Accesso container	Shell come root nel container	Nessuno ancora: l'attaccante e' confinato nel container
3. Escape	Abuso del socket Docker per montare la confusione	Confine dell'isolamento del container: da container a host
4. Condizione finale	Lettura/scrittura come root sull'host	Controllo dell'host e di tutti i container del daemon

Effetto di interrompere o mitigare ogni tappa

- **Correggere la tappa 1** (lettura entro i limiti, protezioni attive): niente shell nel container, la catena si ferma anche col socket montato.
- **Rimuovere la misconfigurazione della tappa 3** (socket non montato): il buffer overflow resta grave ma l'attaccante rimane confinato nel container.
- **Least privilege sulle tappe 2/4** (shell non root, daemon rootless): anche col socket, impatto sull'host ridotto o nullo.

La catena richiede tutte le tappe: spezzarne una qualsiasi la interrompe. Questo motiva la difesa in profondita'.

5. Riproducibilita'

Ambiente costruito dai soli file inclusi, con script dedicati:

```
bash scripts/setup.sh      # pull alpine, pianta la flag host, build e avvio
bash scripts/demo.sh      # baseline + catena completa (bof -> shell -> escape)
bash scripts/cleanup.sh    # smonta, rimuove immagine e flag host
```

Il README contiene i comandi manuali equivalenti per Windows. Nessuna interazione con sistemi esterni.

6. Conclusioni

La catena mostra come una vulnerabilit  di memoria in un'applicazione, unita a una misconfigurazione infrastrutturale, porti dalla compromissione di un singolo processo al controllo completo dell'host. Il buffer overflow fornisce l'accesso iniziale (shell nel container); il socket Docker montato trasforma quell'accesso in controllo dell'host. La difesa pi  efficace combina la correzione della vulnerabilit  applicativa, la rimozione del socket e il least privilege, secondo la difesa in profondit .

Fonti complete in references.md.